Vidarbha Youth Welfare Society's

Prof. Ram Meghe Institute of Technology & Research Badnera, Amravati (M.S.) 444701

(An Autonomus Institute)

Laboratory Manual

Semester IV

Subject: - Android Application Development

Lab (Subject code: -4SX09)

(Academic Year: 2025-2026)

Department of Computer Science & Engg -Internet of Things

Accredited by NAAC A+

Website: www.mitra.ac.in

Vision and Mission of Institute and Department

Vision of the Institute

To become a pace-setting Centre of excellence believing in three universal values namely Synergy, Trust and Passion, with zeal to serve the Nation in the global scenario.

Mission of the Institute

M1: To achieve the highest standard technical education through state of the art pedagogy and enhanced industry institute linkage.
M2: To inculcate the culture of research in core and emerging areas.
M3: To strive for overall development of studies so as to nurture ingenious technocrats as well as responsible citizens.

Vision and Mission of Department of Computer Science and Engineering(IOT)

Vision of the Department

To achieve excellence in the field of Computer Science & Engineering and Internet of Things through dedication & collaboration, so as to meet global standards in education, research and industrial applications and for nurturing young professionals & entrepreneurs with strong belief in social & ethical values for progress of the society.

Mission of the Department

M1: To provide state of the art facilities in order to achieve excellence in teaching – learning, innovative activities, entrepreneurship and research.
M2: To frame curricula and syllabi based on technological needs of industry along with due consideration for moral, ethical and social values, for nation building.
M3: To encourage inter-disciplinary projects and research through industry- institute linkages so as to bridge the gap between industry and academia.
M4: To develop technology-based solutions for relevant societal problems such as environment, health, etc. and impart experiential learning through inter disciplinary/ collaborative research.

2.Program educational objective (PEO's), Program outcomes (PO's) and Program Specific Outcomes (PSO's)

- **Program educational objective (PEO's)**

PEO1: Provide students with opportunities to acquire the ability to apply the fundamentals of mathematics, sciences and engineering to quantitatively analyze engineering problems.

PEO2: Provide students with opportunities to develop industry oriented innovative design skills, including the ability to formulate problems, to think creatively, to synthesize information, and to communicate effectively.

PEO3: Provide students with opportunities to develop the ability to use modern experimental techniques; collect, analyze, and interpret experimental data; and effectively communicate the results.

PEO4: Provide opportunities to prepare students with the diverse skills needed to be successful and competent engineers and/ or to pursue higher studies.

- **Program Outcomes (PO's)**

Engineering Graduate will be able to:
PO1. Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
PO2. Problem analysis: Identify, formulate, review research literature, and analyse complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
PO3. Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
PO4. Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
PO5. Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.
PO6. The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
PO7. Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
PO8. Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
PO9. Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
PO10. Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
PO11. Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
PO12. Life-long learning: Recognize the need for, and have the preparation and ability to engage in

independent and life-long learning in the broadest context of technological change.

- **Program Specific Outcomes (PSO's)**

PSO1: Develop different problem solving skills in order to solve industrial problems based on principles and concepts of CSE & IoT.

PSO2: Comprehend various application development tools and their implementation for IoT applications.

PSO3: To design and implement CSE and IoT based solution for technology based industrial problems

1. Android Application Development Lab (4SX09)

Course Number and Title:	Android Application Development Lab
Credits:	1(Lab)
Name of Faculty:	Prof. P.M. Dundale Prof. A.S. Chaudhari Prof. D.P.Paraskar Prof. R.P. Sawarkar
Course Type:	Lab
Compulsory/ Elective:	Compulsory
Teaching Methods:	Laboratory: 2 Hrs/ week.

Course Assessment:

Grading Policy:

Grades	Marks
A+	10
A	9
B+	8
B	7
C+	6
C	5
D	4

Course Outcomes:

After Successful completion of this course, the student will be able to:

Sr. No.	Course Outcome
4SX09.1	Describe the structure of Android applications and the role of various components such as Activities, Intents, and Services.
4SX09.2	Develop basic Android applications using appropriate UI elements, layouts, and Android APIs.
4SX09.3	Test and debug Android applications and analyze their performance for improved user experience.

Course Learning Objectives:

Mapping of Course Outcomes with Program Outcomes

Course	PO1	PO2	PO3	PO4	PO5	P08	PO9	P010	PO11	PO12
4SX09.1	1	1	1	1	--	--	--	--	--	1
4SX09.2	--	--		1	1	--	--	--	1	--
4SX09.3	--	1	--			1	1	1	1	--

Correlation levels: 1. Low 2. Medium 3.High

Mapping of Course Outcomes with Program Specific Outcomes

Course	PSO 1	PSO 2	PSO 3
4SX09.1	1	1	1
4SX09.2	--	1	1
4SX09.3	--	1	1

Correlation levels: 1. Low 2. Medium 3.High

Gradation Policy for Laboratory Assessment of each Practical for B.Tech Programme

The practicals are of three types –

1. That involves lab equipment such as oscilloscopes, component development systems, signal generators (Test bench)
2. That involves Computers (Simulation)
3. That involves Field work (Community Engagement Project/ Field Project (CEP/FP), mini project, capstone projects etc.)

Following are the components involved in the gradation of the practicals except Community Engagement Project/ Field Project (CEP/FP), mini project, capstone projects etc. by the respective teachers
P1: Attempt and/or attention for execution of Practical task.
P2: Competency (Skill/ability) for performing particular task.
P3: Performance and timely submission of Practical records
P4: Ability to analyze results and draw conclusions.
P5: Question and Answers / Viva-Voce.

Grade	P1	P2	P3	P4	P5
A+	Performance in First attempt; exceptional attention to task	Able to apply skill effectively	Excellent performance & submission of practical record along with results and conclusion by consecutive practical day.	Highly insightful analysis with accurate, logical, and well-developed conclusions	Appropriate and accurate answers
A	Performance in First attempt; excellent attention to task	Good skill application	Effective performance & submission of practical record along with results and conclusion by consecutive practical day.	Clear, accurate analysis with well-reasoned conclusions	Relevant and well-explained answers
B+	Performance in First attempt; above average attention to task	Satisfactory skill application	Good performance & submission of practical record one week late then timely date of submission	Generally correct analysis with mostly appropriate conclusions	Mostly appropriate answers
B	Performance in second attempt; average attention to task	Minimal understanding skill application	Acceptable performance & submission of practical record one week late then timely date of submission	Minimal analysis with partially correct or incomplete conclusions	Partially appropriate answers

C+	Performance in second attempt; below average attention to task	Somewhat skill application	Inconsistent performance & submission of practical record two week late then timely date of submission	Limited analysis with conclusions lacking clarity or accuracy.	Limited but somewhat relevant answers
C	Performance in third attempt or more; somewhat attentive	Severe difficulty in applying skills	Poor performance & submission of practical record two week late then timely date of submission	Minimal analysis with weak or unsupported conclusions	Weak or incomplete answers
D	Performance in third attempt or more; least attentive	Able to apply skills with help from faculty	Very poor performance & very late submission of practical record	Unable to analyze results; conclusions incorrect or missing	Incorrect or irrelevant answers

Mapping of Practicals gradation with marks

Grades	Marks
A+	10
A	9
B+	8
B	7
C+	6
C	5
D	4

List of Experiments

Department of CSE-IOT

Subject: - Android Application Development Lab **Semester: -4th**

Subject Code: -4SX09

Sr No	List of Practical
1.	1.A) To install Android Studio on a windows machine for Android app development.
	1.B) To understand what a Flutter plugin is and to install the Flutter plugin and Flutter SDK in Android Studio for developing Flutter applications.
2.	To develop and run a simple "Hello World" mobile application using Flutter and display text on the screen.
3.	To develop a Flutter application that changes the displayed text dynamically when a button is clicked using the setState() method in a Stateful widget.
4.	To design a Flutter application that changes the screen background color dynamically when a button is pressed using a Stateful widget and setState() method.
5.	To create a Flutter application that toggles the visibility of text on the screen using a button, by changing the state variable in a Stateful widget.
6.	To develop a Flutter application that updates the label text of a button dynamically when it is pressed using a Stateful widget and state management.
7.	To design a Flutter application that accepts text input from the user through a TextField and immediately displays it on the screen using state management.
8.	To create a Flutter application that demonstrates how a checkbox can be selected or deselected and how its state can be used to display different text dynamically.
9.	To develop a Flutter application that allows the user to select an option using radio buttons and display the selected value dynamically on the screen.
10.	To develop a Flutter application that displays a list of items and shows the selected item when the user clicks on it.
11.	To develop a Flutter application that dynamically switches between Row and Column layout based on screen width.
12.	To navigate between two screens in a Flutter application using Navigator.push() and Navigator.pop().
13.	To develop a Flutter application that switches between light and dark themes dynamically.
14.	To animate size and color transitions in Flutter using the AnimatedContainer widget.
15.	To display multiple items using a grid layout in Flutter with the GridView widget.
16.	To create a navigation drawer in a Flutter application.

Practical No.- 1 (A)

Aim: To install Android Studio on a windows machine for Android app development.

Required Tools and Software:

1. Android Studio Installer

Theory:

Android Studio:

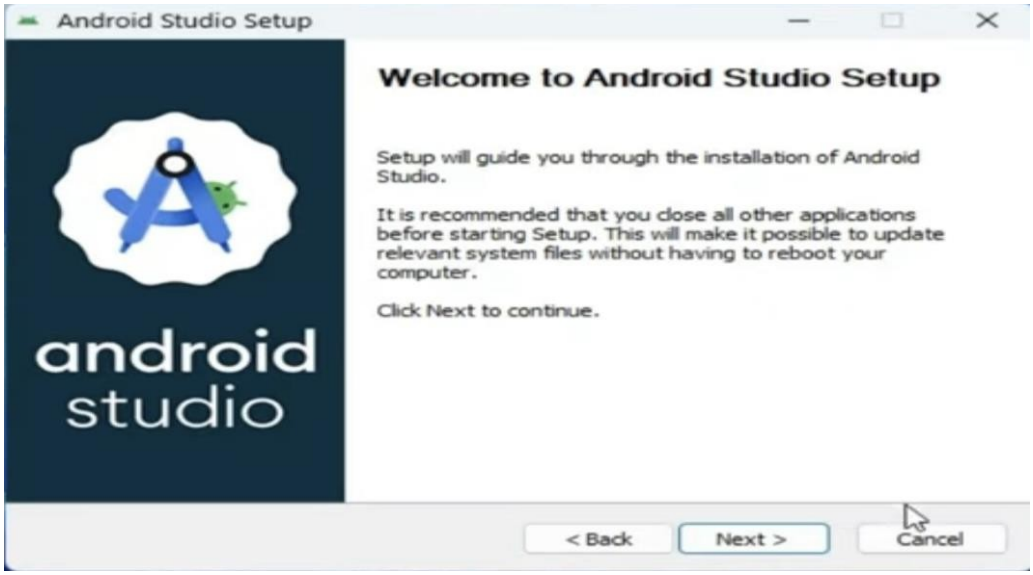
Android Studio is the official integrated development environment (IDE) for building Android applications. It is built on IntelliJ IDEA and provides tools for code editing, debugging, performance tuning, and UI design. Android Studio supports Kotlin, Java, and C++ for Android app development, and it includes a built-in Android Emulator for testing apps. It also comes with essential tools like the Android SDK, Android Virtual Device (AVD) Manager, and various testing frameworks, making it an essential tool for Android developers.

Java SDK (Software Development Kit):

The Java SDK (often referred to as JDK - Java Development Kit) is a software development kit used to build Java applications. It includes the Java Runtime Environment (JRE), an interpreter/loader, a compiler (javac), an archiver (jar), a documentation generator (javadoc), and other tools necessary for Java development. The JDK is essential for running and compiling Java code, and it's required to develop Android apps using Java. Android Studio uses the JDK to build and run Java-based Android applications.

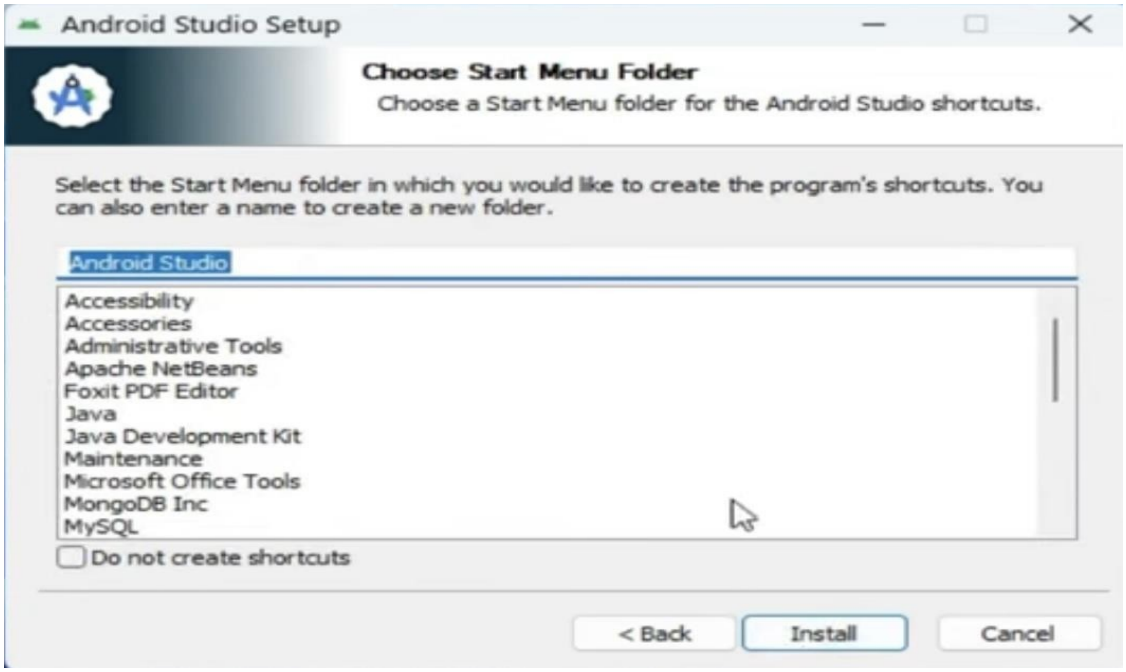
Steps for Installation:

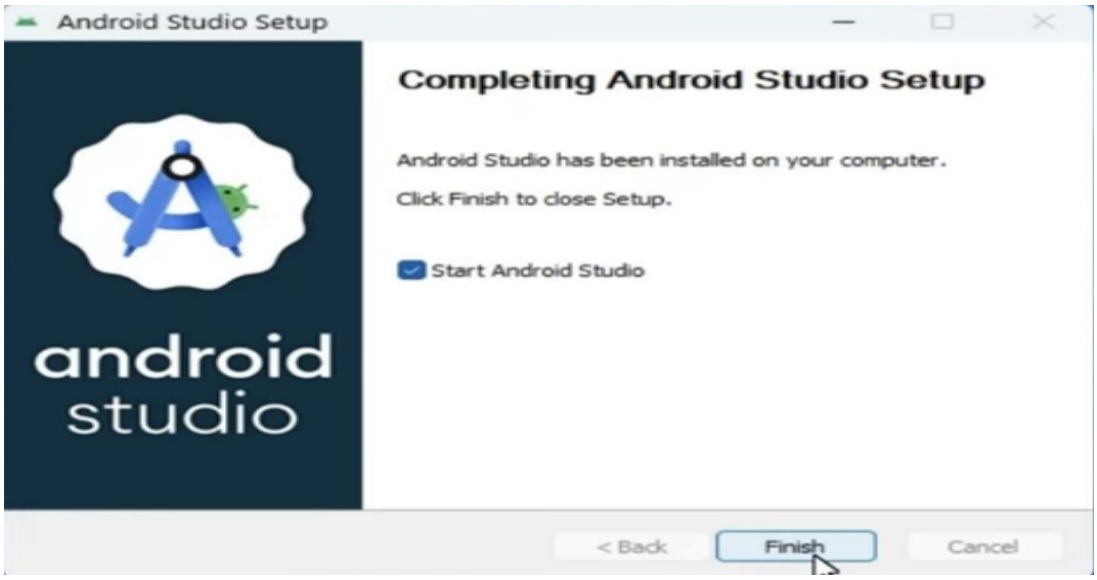
1) Browse to download android studio latest version. You will be redirect to welcome page.

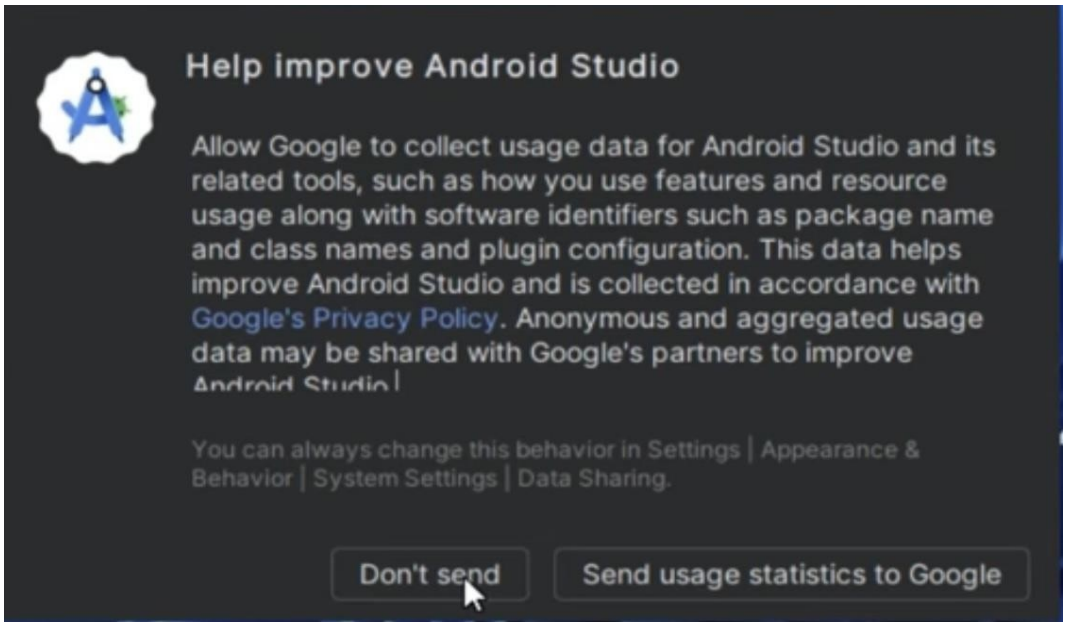


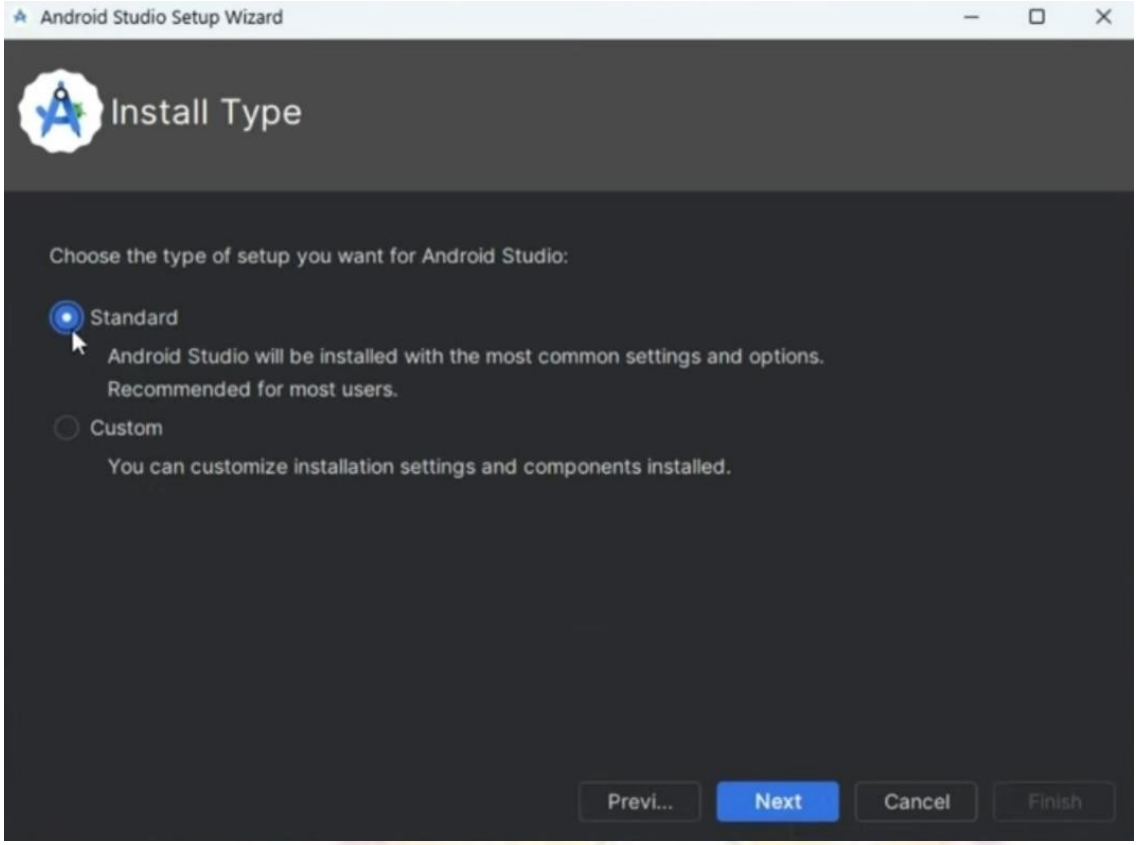
1) Check the android virtual device check box, to access the virtual tools.

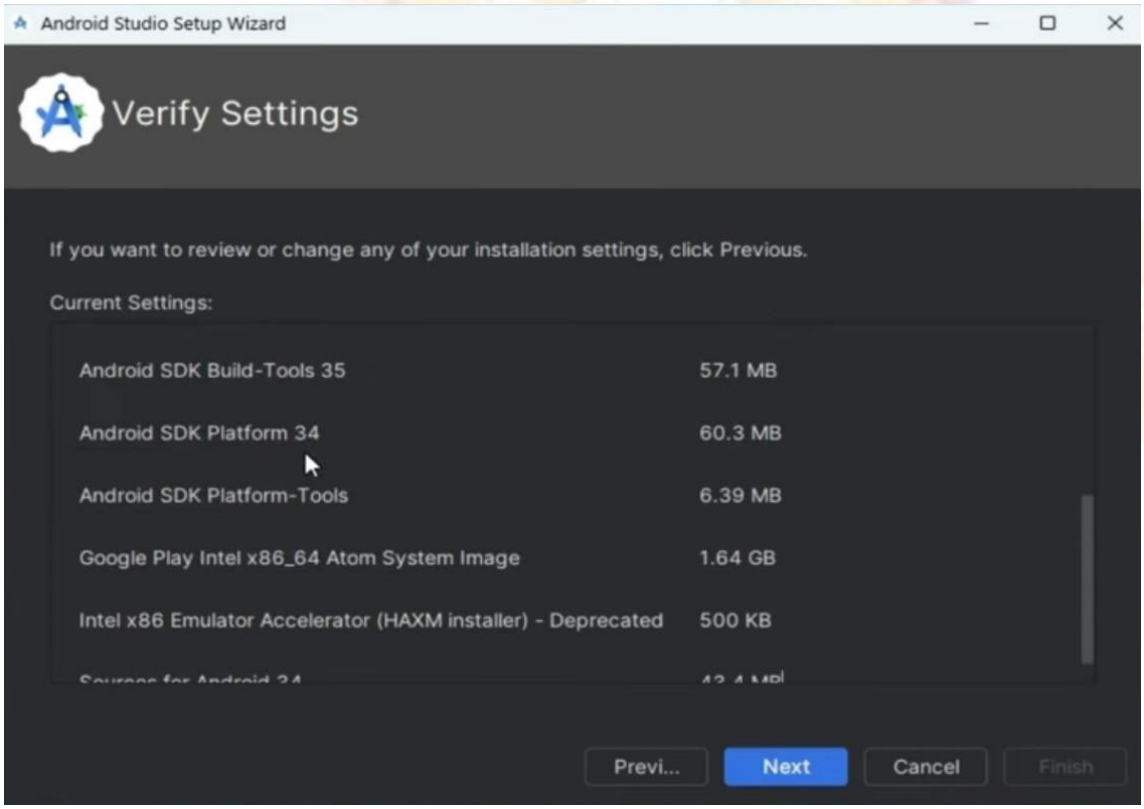
2) The java development kit must be available in the list. And let it installed.



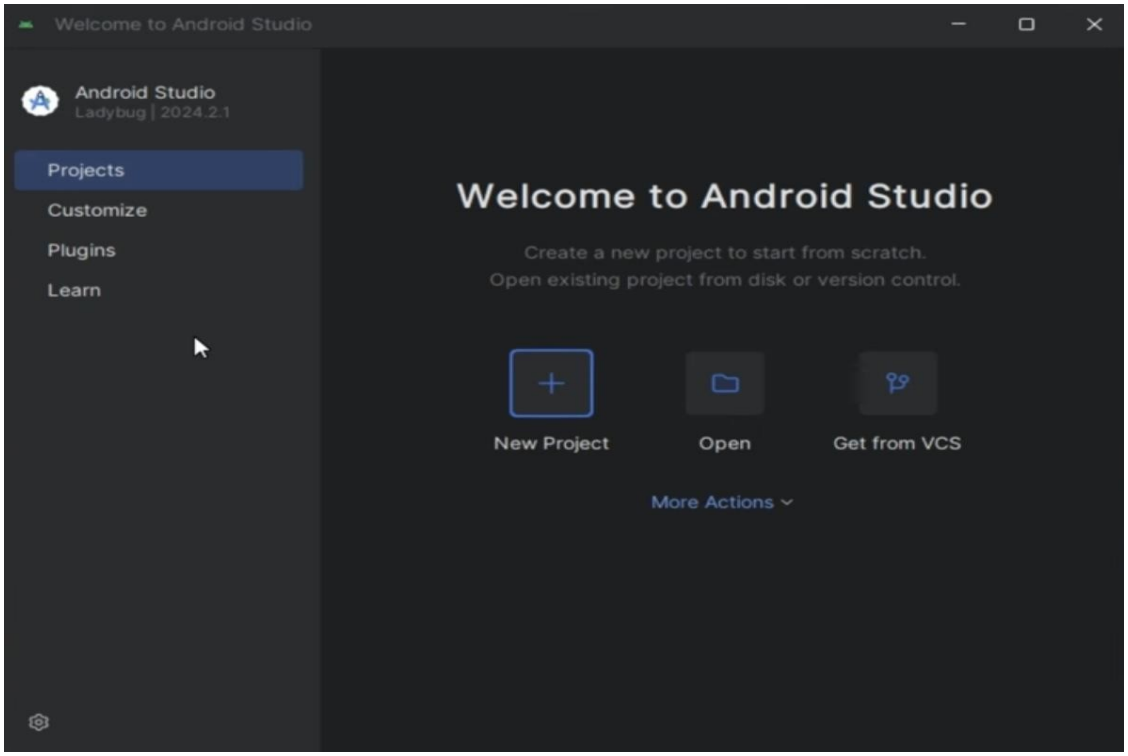

Android Studio Setup
Completing Android Studio Setup
Android Studio has been installed on your computer.
Click Finish to close Setup.
Start Android Studio
android
studio
< Back
Finish
Cancel

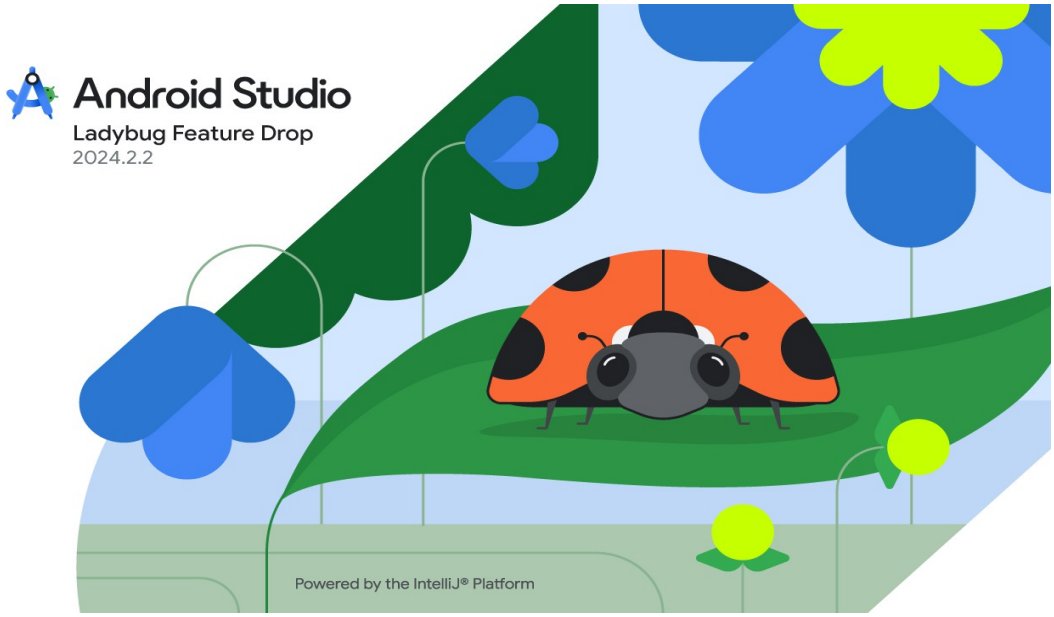

Help improve Android Studio
Allow Google to collect usage data for Android Studio and its related tools, such as how you use features and resource usage along with software identifiers such as package name and class names and plugin configuration. This data helps improve Android Studio and is collected in accordance with Google's Privacy Policy. Anonymous and aggregated usage data may be shared with Google's partners to improve Android Studio.
You can always change this behavior in Settings | Appearance & Behavior | System Settings | Data Sharing.
Don't send
Send usage statistics to Google


Android Studio Setup Wizard
Install Type
Choose the type of setup you want for Android Studio:
Standard
Android Studio will be installed with the most common settings and options.
Recommended for most users.
Custom
You can customize installation settings and components installed.
Previ...
Next
Cancel
Finish


Android Studio Setup Wizard
Verify Settings
If you want to review or change any of your installation settings, click Previous.
Current Settings:
Android SDK Build-Tools 35 57.1 MB
Android SDK Platform 34 60.3 MB
Android SDK Platform-Tools 6.39 MB
Google Play Intel x86_64 Atom System Image 1.64 GB
Intel x86 Emulator Accelerator (HAXM installer) - Deprecated 500 KB
Previ...
Next
Cancel
Finish

And let the components download.





Conclusion:

Questions:

1) List and explain the features the Android studio Ladybug feature drop provides.

Practical No.- 1 (B)

Aim: To understand what a Flutter plugin is and to install the Flutter plugin and Flutter SDK in Android Studio for developing Flutter applications.

Theory:

What is Flutter?
Flutter is an open-source UI framework developed by Google for building cross-platform applications using a single codebase. Using Flutter, we can develop Android, iOS, Web, and Desktop applications.
Flutter uses the Dart Programming language.

What is a Flutter Plugin in Android Studio?

A Flutter plugin is an extension for Android Studio that provides tools and features required to develop Flutter applications.

Why Flutter Plugin is Needed?

The Flutter plugin helps Android Studio to:

- Create Flutter projects
- Write and edit Dart code

- Provide code suggestions and error checking
- Run and debug Flutter apps
- Connect Flutter SDK with Android Studio

Without the Flutter plugin, Android Studio cannot recognize Flutter projects.

What is Flutter SDK?
The Flutter Software Development Kit (SDK) is a collection of:

- Flutter framework libraries
- Dart SDK
- Command-line tools
- Predefined widgets and packages

The Flutter SDK is required to compile, run, and test Flutter applications.

Procedure: Part A: Installing Flutter Plugin in Android Studio

Step 1: Open Android Studio

- Launch Android Studio
- On the welcome screen, click Plugins

  (or File → Settings → Plugins)

Step 2: Search Flutter Plugin

- In the Marketplace, type Flutter
- Click Install on the Flutter plugin

Step 3: Install Dart Plugin

- Android Studio will ask to install Dart plugin
- Click Install
- Dart is mandatory because Flutter uses Dart language

Step 4: Restart Android Studio

- After installation, click Restart IDE
- Android Studio will restart with Flutter support enabled

Part B: Installing Flutter SDK

Step 1: Download Flutter SDK

- Visit official Flutter website

  https://flutter.dev

- Click Get Started

- Select your operating system (Windows / macOS / Linux)
- Download Flutter SDK ZIP file

Step 2: Extract Flutter SDK

- Extract the downloaded ZIP file
- Recommended location:
    - Windows: C:\flutter
    - macOS/Linux: /Users/username/flutter

Do not place Flutter SDK inside Program Files.

Step 3: Add Flutter SDK Path in Android Studio

- Open Android Studio
- Go to File → Settings
- Navigate to Languages & Frameworks → Flutter
- In Flutter SDK Path, browse and select:

C:\flutter

- (or the extracted folder path)
- Click Apply → OK

Part C: Verify Flutter Installation

Step 1: Open Terminal in Android Studio

- Click Terminal (bottom panel)

Step 2: Check Output

- This command checks:
    - Flutter SDK
    - Dart SDK
    - Android Studio
    - Emulator or connected device

Conclusion:
Questions

1. What is Flutter?
2. Why is Flutter plugin required in Android Studio?
3. What is the role of Dart in Flutter?
4. What is Flutter SDK?
5. What does flutter doctor command do

Practical No.- 2

Aim: To develop and run a simple "Hello World" mobile application using Flutter and display text on the screen.

Required Tools and Software:

- Android Studio (latest version)
- Flutter SDK
- Dart SDK (included with Flutter)
- Android Emulator / Physical device with USB debugging enabled

Theory:

Flutter is an open-source UI toolkit developed by Google that allows developers to build cross-platform mobile, web, and desktop applications from a single codebase using the Dart Programming language.

A Flutter application is composed of widgets, which are visual building blocks like text, buttons, and layouts.

In this Program, the key widget MaterialApp provides the app design structure, Scaffold gives layout elements like AppBar and body, and Text widget prints "Hello World".

This experiment demonstrates how Flutter renders UI using widgets to create basic applications.

Algorithm::

1. Start the Program.
2. Import the Flutter material package to use widgets.
3. Define the main() function and call runApp() to start the Flutter application.
4. Create a StatelessWidget class (MyApp) which returns UI of the app.
5. Inside build() method, return a MaterialApp widget to provide basic app structure.
6. Disable the debug banner using debugShowCheckedModeBanner: false.
7. Create a Scaffold widget to provide app layout structure.
8. Add an AppBar with the title "Flutter Hello World".
9. Set the body of the screen with a Center widget.
10. Display "Hello World" text with font size 30 using the Text widget.
11. End the Program.

Program:
Output:
Conclusion:
Questions

1. What is Flutter and why is it used for mobile app development?
2. What is the role of the main() function in Flutter?
3. Define widget. What is the difference between StatelessWidget and StatefulWidget?
4. What does MaterialApp do in a Flutter project?

Practical No.- 3

Aim: To develop a Flutter application that changes the displayed text dynamically when a button is clicked using the setState() method in a Stateful widget.

Software Requirements:

- Android Studio / VS Code
- Flutter SDK installed
- Dart SDK (included with Flutter)

Hardware Requirements:

- Windows/Linux/macOS system
- Minimum 4GB RAM
- Android Emulator or Physical Mobile Device

Theory:

Flutter is an open-source UI development framework developed by Google that allows developers to build natively compiled mobile, web, and desktop applications using a single codebase. It uses the Dart Programming language, which supports both object-oriented and reactive Programming. In Flutter, widgets are the building blocks of the application. Everything visible on the screen is a widget—text, buttons, layouts, and even the entire app structure itself.

There are two types of widgets in Flutter: Stateless and Stateful. A Stateless widget does not change once it is built; its values remain constant. A Stateful widget, on the other hand, can change its appearance during runtime based on user actions or data updates. In this experiment, we use a Stateful widget because the text on the screen must update when the button is clicked.

The concept of state management plays a key role in controlling UI updates. Flutter uses a reactive model, meaning whenever the state changes, only the affected widget subtree is rebuilt. The method setState() is one of the simplest mechanisms to notify Flutter that the UI must be refreshed to reflect new data. When the user presses the button, the message variable changes from “Welcome” to “Button Clicked”. Calling setState() signals Flutter to redraw the widget tree, which updates the text displayed.

The layout of the application is defined using widgets like MaterialApp, Scaffold, AppBar, and Column. The ElevatedButton triggers the interaction. By placing widgets inside a Column and centering them using Center, we ensure proper alignment. This experiment demonstrates event handling, widget hierarchy, and dynamic UI behavior—basic but essential concepts required to build interactive Flutter applications.

Algorithm:

1. Start the Flutter application and import the required Material package.
2. Create the main function and call runApp() to start the app.
3. Define a Stateful widget class MyApp.

4. Inside its state class, declare a string variable named message and initialize it with "Welcome".
5. In the build method, return a MaterialApp containing a Scaffold.
6. Add an AppBar to display the title of the application.
7. Inside the body, place a Column widget aligned to the center.
8. Display the message using a Text widget.
9. Insert an ElevatedButton widget below the text.
10. Assign an onPressed function to the button.
11. Inside onPressed, call setState() to modify the value of message.
12. Update the text from "Welcome" to "Button Clicked".
13. When setState() executes, the UI rebuilds automatically.
14. Display the updated message on the screen.
15. Stop execution.

Program:
Output:

Conclusion:

Questions:

1. Why must we use setState() in a Stateful widget and not just change the variable value?
2. What will happen if we remove setState() and only update message?
3. Can a Stateless widget also change text? Why or why not?

Practical No- 4

Aim: To design a Flutter application that changes the screen background color dynamically when a button is pressed using a Stateful widget and setState() method.

Requirements:

Software Requirements

- Flutter SDK with Dart
- Android Studio / VS Code
- Android Emulator or Real Device

Hardware Requirements

- PC/Laptop with minimum 4GB RAM
- USB cable (if using physical device)

Theory:

Flutter provides a reactive UI system in which the user interface automatically updates whenever the underlying state changes. Widgets in Flutter are broadly categorized into Stateless and Stateful widgets. A Stateless widget does not reflect runtime changes, whereas a Stateful widget is designed to store and modify state values when events occur, such as user interactions.

In this experiment, we use a Stateful widget to change the background color of the screen. The Stateful widget contains a variable bgColor that initially stores the color Colors.white. When the user taps the "Change Color" button, the onPressed callback executes and updates the variable to a new color value. To apply this change on the UI, we call setState(), which informs Flutter that the state has changed. This triggers the rebuilding of the widget tree and results in updating the screen background to the new color.

The application uses MaterialApp and Scaffold, which are fundamental widgets in Flutter's Material Design framework. The Scaffold widget provides a base structure that includes AppBar, body area, and background color support. Changing the backgroundColor attribute allows us to modify the entire screen color. This Program illustrates the basic principle of event-driven UI updates, highlighting how user actions directly impact application visuals through state manipulation.

Algorithm:

1. Start the application by importing Flutter's Material package.
2. Create a Stateful widget named ColorApp.
3. Define a state variable bgColor and initialize it to white.
4. Inside the build() method, return a MaterialApp with Scaffold.
5. Set the Scaffold's backgroundColor to bgColor.
6. Add an AppBar displaying the title "Color Change".
7. Place an ElevatedButton in the center of the screen.
8. Assign an onPressed function to the button.
9. Inside the function, call setState().
10. Change the value of bgColor to a different color (lightBlue).
11. Rebuild the UI to reflect the updated color.

12. Display updated background color on screen.
13. End Program.

Program:
Output:
Conclusion:
Question:

1. What difference would occur if we changed the value of bgColor without calling setState()?
2. Why must this Program be written using a Stateful widget instead of Stateless widget?
3. What parts of the UI are rebuilt after calling setState() in this Program?

Practical No- 5

Aim: To create a Flutter application that toggles the visibility of text on the screen using a button, by changing the state variable in a Stateful widget.

Reequipments:

Software Requirements

- Flutter SDK & Dart
- Android Studio or VS Code
- Android Emulator or Physical Device
- Laptop/PC with minimum 4 GB RAM

Theory:

Flutter follows a reactive UI model, where screen elements update automatically when the underlying data (state) changes. To support dynamic updates, Flutter provides two types of widgets: Stateless and Stateful. A Stateless widget is static and cannot update its UI after rendering. A Stateful widget allows the user interface to redraw based on state changes.

This experiment demonstrates runtime visibility control of a text element. A Boolean variable isVisible is used to decide whether the text widget should be displayed. Flutter provides conditional rendering using if() statements directly in the widget tree. When the variable is true, the text appears on screen; when false, it disappears.

The ElevatedButton triggers a call to setState(), which flips the boolean value using !isVisible. The ! operator ensures toggling—true becomes false and false becomes true. Since the UI depends on isVisible, the build method executes again automatically and removes or adds the text widget accordingly. The layout uses Column inside the Center widget for proper alignment. This experiment clearly illustrates events, widget tree rebuilding, conditional layout rendering, and fundamental user interaction in Flutter.

Algorithm:

1. Start the Program by importing the Material package.
2. Create a Stateful widget ToggleTextApp.
3. Declare a boolean variable isVisible initialized to true.
4. Inside the build() method, return a MaterialApp with Scaffold.
5. Add an AppBar displaying “Show / Hide Text”.
6. Use Center and Column widgets to position elements vertically.
7. Check the value of isVisible.
8. If true, display the text “Hello Flutter”.
9. Add an ElevatedButton below the text.
10. Assign an onPressed function to the button.
11. Inside onPressed, call setState() and toggle isVisible.
12. Rebuild UI and update visibility based on new value.
13. End execution

Program:
Output:
Conclusion:
Question:

1. Why do we use !isVisible instead of assigning a fixed true/false value?
2. What happens if we remove the if(isVisible) condition but still toggle the variable?
3. How can we modify this Program to show different text each time instead of hiding it?

Practical No- 6

Aim: To develop a Flutter application that updates the label text of a button dynamically when it is pressed using a Stateful widget and state management.

Requirements:

- Flutter SDK with Dart
- Android Studio or VS Code
- Android emulator / Physical mobile device
- Laptop/PC with minimum 4GB RAM
- USB cable (if using phone), Internet (for Flutter setup)

Theory:

Flutter is based on a reactive Programming model where the user interface updates whenever the application's state changes. Widgets are the core building blocks of the UI, and they are categorized as Stateless and Stateful. A Stateless widget paints the UI only once and cannot change dynamically. A Stateful widget allows developers to store and modify values over time and refresh the visible layout when the state changes.

In this experiment, the application uses a Stateful widget that contains a string variable representing the button text. Initially, this variable is set to "Submit". The UI displays this value on the button. When the user taps the button, the onPressed() callback function executes and modifies the variable to "Submitted". To reflect this change on the screen, setState() is invoked. This function notifies Flutter that the UI needs rebuilding because the underlying data has changed.

Flutter then re-executes the build() method and updates the button's label to the new value stored in the variable. This experiment demonstrates event handling, dynamic label updates, widget tree rebuilding, and basic state management—key concepts required for interactive application development in Flutter.

Algorithm:

1. Import Flutter's Material package.
2. Define the main function and call runApp().
3. Create a Stateful widget named ButtonTextApp.
4. Declare a state variable buttonText and initialize it to "Submit".
5. In the build() method, return a MaterialApp with Scaffold.
6. Add an AppBar displaying the title "Button Text Change".
7. Place an ElevatedButton in the center of the screen.
8. Display the value of buttonText inside the button.
9. Assign an onPressed function to the button.
10. Inside the function, use setState() to change buttonText to "Submitted".
11. Flutter rebuilds the widget tree automatically.
12. Updated button text appears on the screen.
13. End Program.

Program:
Output:
Conclusion:
Question:

1. Why must we update the button text inside setState() instead of directly changing the variable?
2. What will happen if the user presses the button multiple times in this Program?
3. How can you modify this Program so that pressing the button toggles between Submit and Submitted?

Practical No- 7

Aim: To design a Flutter application that accepts text input from the user through a TextField and immediately displays it on the screen using state management.

Requirements:

- Flutter SDK & Dart
- Android Studio / VS Code
- PC/Laptop with minimum 4 GB RAM
- USB cable (optional for real device)

Theory:

Flutter uses a reactive framework where widgets automatically rebuild when the internal state changes. In interactive applications, user input is captured using widgets such as the TextField. TextField allows users to enter text, numbers, or characters, and it includes built-in event handlers to track typing. Unlike static widgets, TextField generates changes dynamically as the user types.

This experiment demonstrates how to capture input from a TextField and reflect it instantly on the screen. A Stateful widget maintains a variable named name, initially empty. The onChanged callback of TextField retrieves every character typed by the user. Inside this callback, the setState() function is used to update the variable name. Calling setState() notifies Flutter that the internal data has changed, causing the build() method to rerun and refresh the UI.

The text "Hello $name" displayed on the screen updates in real time because it reads the current value of name during every UI rebuild. This experiment highlights the concepts of data binding, live UI updates, and the importance of state variables in Flutter. It also shows how Flutter automatically handles widget re-rendering, allowing developers to build highly interactive applications with minimal code.

Algorithm:

1. Start by importing Flutter's Material package.
2. Create a Stateful widget named InputDisplayApp.
3. Declare a string variable name and initialize to an empty string.
4. Build a MaterialApp with a Scaffold.
5. Add an AppBar showing the title "TextField Input".
6. Use Padding to add spacing around the content.
7. Insert a TextField widget that allows users to type text.
8. Use the onChanged(value) event to capture user input.
9. Call setState() and update the variable name with the new value.
10. Add a Text widget that displays "Hello $name".
11. When state changes, Flutter rebuilds and updates the displayed text.
12. End Program.

Program:Output:

Conclusion:

Question:

1. What is the purpose of using onChanged() instead of reading the TextField value only once?
2. What will happen if setState() is removed inside the onChanged() function?
3. How can you modify the Program to display the text only after a button press instead of live typing?

Practical No- 8

Aim: To create a Flutter application that demonstrates how a checkbox can be selected or deselected and how its state can be used to display different text dynamically.

Requirements:
- Flutter SDK & Dart
- Android Studio / VS Code
- Emulator or Android Phone
- PC/Laptop with 4GB RAM or above
- USB cable (optional for device testing)

Theory:
Flutter's user interface is built from widgets that respond to changes in state. A checkbox is a common UI component used to represent binary choices—true or false, yes or no, checked or unchecked. To make checkboxes interactive, the application must use a Stateful widget, as the value changes when users interact with it.

In this experiment, a boolean variable isChecked is used to track the state of the checkbox. Initially, its value is false, meaning the checkbox appears unchecked. The Checkbox widget displays this value via its value property. When the user taps the checkbox, the onChanged function triggers and returns a boolean value (true or false). To update the UI, setState() is called to assign this new value to the isChecked variable.

Flutter automatically rebuilds the widget tree, and both the checkbox UI and the text below it change according to the updated state. The text "Checked" appears when the value is true and "Unchecked" appears when false. This experiment illustrates interactive input handling, conditional UI rendering, and the importance of state variables in dynamic Flutter widgets.

Algorithm:
1. Import Flutter Material library.
2. Create a Stateful widget named CheckBoxApp.
3. Declare a boolean variable isChecked with initial value false.
4. In the build() method, return a MaterialApp and Scaffold.
5. Add an AppBar with title "Checkbox Example".
6. Use a Column widget placed inside Center for vertical layout.
7. Insert a Checkbox widget with its value set to isChecked.
8. Use onChanged() callback to receive updated checkbox state.
9. Inside onChanged, call setState() and update isChecked.
10. Below the checkbox, display "Checked" if isChecked is true and "Unchecked" if false.
11. Flutter rebuilds UI automatically reflecting the new state.
12. End Program.

Program:
Output:
Conclusion:
Question:
1. Why must the checkbox be placed inside a Stateful widget instead of a Stateless widget?
2. What will happen if the value! operator is removed in the onChanged callback?
3. How can this Program be modified to handle multiple checkboxes at once?

Practical No- 9

Aim: To develop a Flutter application that allows the user to select an option using radio buttons and display the selected value dynamically on the screen.

Requirements:

- Flutter SDK & Dart
- Android Studio / VS Code
- Emulator or Physical Device
- Laptop/PC with minimum 4GB RAM

Theory:

Radio buttons are widely used UI components for selecting one option among multiple choices. Unlike checkboxes that support multiple selections, radio buttons enforce mutual exclusivity—only one option can be active at a time. Radio buttons work by grouping related choices using a shared state variable called groupValue, which determines which item is currently selected.

In Flutter, the Radio and RadioListTile widgets provide built-in support for radio button interfaces. value identifies the individual radio button, while groupValue tracks the current selection. When the user taps a radio button, the onChanged() callback triggers and returns the selected value. To update the UI, setState() stores the new choice inside a state variable and rebuilds the widget tree. This experiment uses a String variable choice initialized to "Male". When a user selects "Female", the value changes dynamically and the text "Selected: Female" appears. This demonstrates reactive UI updates, state handling, and mutually exclusive inputs—important concepts in form design and data collection within Flutter applications.

Algorithm:

1. Import the Material package.
2. Create a Stateful widget named RadioApp.
3. Declare a String variable choice with default value "Male".
4. In the build() method, return a MaterialApp with Scaffold.
5. Add an AppBar titled “Radio Button”.
6. Place RadioListTile widgets inside a Column.
7. Set value of each RadioListTile to "Male" or "Female".
8. Set groupValue equal to the variable choice.
9. Implement onChanged(value) to detect selection.
10. Inside onChanged, call setState() and update choice.
11. Display selected value in a Text widget at the bottom.
12. UI refreshes automatically after state change.
13. End Program.

Program:

Output:

Conclusion:

Question:

1. Why do radio buttons require a groupValue while checkboxes do not?
2. How would you modify this code to handle three or more gender options?
3. What happens if two RadioListTile widgets share the same value?

Practical No- 10

Aim: To develop a Flutter application that displays a list of items and shows the selected item when the user clicks on it.

Requirements:
• Android Studio or VS Code
• Flutter SDK installed
• Android Emulator or physical Android device
• Basic computer system

Theory:
Flutter is a framework used for creating cross-platform applications using a single codebase written in Dart.
A ListTile is a common widget used to display text with optional interaction support.
When a user taps on a ListTile, the onTap() callback function is executed.
To update the user interface dynamically, the setState() method is used inside a StatefulWidget.
In this experiment, when a user taps on Item 1 or Item 2, the selected item is stored in a variable and displayed on the screen.

Procedure / Steps:

1. Create a new Flutter project.
2. Open the main.dart file.
3. Replace the existing code with the given Program.
4. Run the app using an emulator or real device.
5. Click on Item 1 or Item 2.
6. Observe that the selected item text changes.

Program:

Output:
Conclusion:
Question:

1. What is ListTile used for?
2. What happens when the user taps on an item?
3. Explain the difference between StatelessWidget and StatefulWidget.

Practical No- 11

Aim: To develop a Flutter application that dynamically switches between Row and Column layout based on screen width.

Requirements:
• Flutter SDK
• Android Studio or VS Code
• Android Emulator or Physical Device
• Basic computer system

Theory:

Flutter provides a feature to build responsive user interfaces using widgets like LayoutBuilder.
Screen sizes vary across devices such as mobiles, tablets, and desktops.
LayoutBuilder gives the size constraints of the available screen and allows developers to change UI structure accordingly.
If the screen width is above a threshold (for example, 600 pixels), a Row layout is displayed.
If the width is less than the threshold, a Column layout is used instead.
This helps to create applications suitable for multiple form factors using one codebase.

Steps / Procedure:

1. Create a new Flutter project.
2. Replace existing code in main.dart with the provided Program.
3. Run the application.
4. View the UI on a small screen (mobile emulator) and observe a vertical Column layout.
5. Resize window or run on tablet mode to observe Row layout.
6. Notice automatic layout switching based on screen width.

Program:

Output:

Conclusion:
Question:

1. What is LayoutBuilder used for in Flutter?
2. Why is responsive design important?
3. What happens when screen width crosses 600 pixels?

Practical No- 12

Aim: To navigate between two screens in a Flutter application using Navigator.push() and Navigator.pop().

Requirements:
• Flutter SDK
• Android Studio or VS Code
• Emulator or Physical Device
• Laptop/Desktop system

Theory:
Navigation refers to moving from one screen (page) of an app to another.
In Flutter, every screen is represented as a widget, usually using Scaffold.
Navigator is a widget that manages a stack of pages.
Navigator.push() adds a new page onto the stack and displays it to the user.
Navigator.pop() removes the topmost page and returns to the previous screen.
MaterialPageRoute is used to define how a page transition occurs.
This push/pop mechanism is similar to function calls where push enters a new context and pop exits it.
Using multiple screens makes apps more interactive and modular.
Navigation is essential in real-world apps like forms, menus, and settings pages.

Algorithm:
1. Start the Flutter application.
2. Create two pages using StatelessWidget classes.
3. Display a button on the first page.
4. When the button is pressed, call Navigator.push() to open the second page.
5. Place a button on the second page to return.
6. When pressed, execute Navigator.pop() to go back to the first page.

Program:
Output:
Conclusion:
Question:

1. What is the purpose of Navigator.push()?
2. What does Navigator.pop() do?
3. What is MaterialPageRoute used for?

Practical No- 13

Aim: To develop a Flutter application that switches between light and dark themes dynamically.

Requirements:

• Flutter SDK
• Android Studio or VS Code
• Android emulator or mobile device
• Basic computer system

Theory:

Flutter supports dynamic theming using ThemeData.
A theme defines the overall look and feel of the app, including colors, typography, and component styles.
MaterialApp provides parameters such as theme and darkTheme to display visual styles.
By using a boolean flag and setState, the app UI can be updated whenever the switch is toggled.
SwitchListTile widget combines a label and a switch control in a single UI line.
When the switch changes state, the UI rerenders with the selected theme.
This approach is widely used in real applications like Chrome, YouTube and system-level dark mode toggles.
Theme switching improves readability, battery usage and nighttime visibility.
Flutter provides prebuilt ThemeData.light() and ThemeData.dark() for quick implementation.

Algorithm:

1. Start the Flutter project.
2. Create a StatefulWidget to manage UI changes.
3. Declare a boolean variable for theme selection.
4. Apply the theme property to MaterialApp based on the variable.
5. Use SwitchListTile to capture user interaction.
6. When the switch is toggled, call setState to change the theme.
7. Observe the UI switching between light and dark themes instantly.
8. To switch between light and dark theme.

Program:
Output:
Conclusion:
Question:

1. What is ThemeData in Flutter used for?
2. Why do we require setState() in this Program?
3. Which widget combines label and switch in one line?

Practical No- 14

Aim: To animate size and color transitions in Flutter using the AnimatedContainer widget.

Requirements:
• Flutter SDK
• Android Studio or VS Code
• Emulator or physical device
• Laptop or desktop system

Theory:
Animation brings smooth visual transitions to UI components.
Flutter provides implicit animations that automatically animate changes to widget properties when they are updated.
AnimatedContainer is one such widget that animates size, shape, color, border and padding changes without requiring an AnimationController.
When the user interacts and values are modified, setState triggers a rebuild and AnimatedContainer smoothly transitions to the new properties over a specified duration.
Implicit animation reduces complexity because the framework handles interpolation automatically.
Animations make mobile applications more interactive, visually appealing and intuitive.
In this Program, both width/height and background color gradually animate over one second when the button is pressed.
By toggling values, continuous animation can be generated with very minimal code.
Algorithm:

1. Create a StatefulWidget to manage dynamic UI changes.
2. Declare size and color variables with initial values.
3. Place an AnimatedContainer widget in the UI.
4. Set duration property to define how long the animation will last.
5. Add a button that toggles size and color.
6. On click, update the properties inside setState.
7. Observe the smooth animated transitions when values change.

Program:

Output:
Conclusion:
Question:

1. What is the purpose of AnimatedContainer in Flutter?
2. How do implicit animations differ from explicit animations?
3. Why is setState necessary for animation?

Practical No- 15

Aim: To display multiple items using a grid layout in Flutter with the GridView widget.

Requirements:
• Flutter SDK
• Android Studio or VS Code
• Emulator or physical device
• Laptop/Desktop system

Theory:

Flutter provides GridView for displaying widgets in a structured grid format.
A grid layout arranges items into rows and columns, useful for galleries, menus, and product listings.
GridView.count allows specifying the number of columns using the crossAxisCount property.
The children parameter contains the widgets that need to be displayed in the grid.
List.generate dynamically creates a list of widgets at runtime, avoiding manual repetition.
Each generated item is placed inside a container with margin, color, and centered text.
Flutter automatically handles scrolling if the content exceeds available screen space.
Grid layouts help in designing clean, tiled UI structures common in modern applications.

Algorithm:

1. Start a new Flutter project.
2. Create a StatelessWidget named GridApp.
3. Inside Scaffold, add a GridView.count widget.
4. Set crossAxisCount to define number of columns.
5. Use List.generate to create items dynamically.
6. Display each item in a styled container.
7. Run the app to view the grid layout.

Program:

Output:
Conclusion:
Question:

1. What is the purpose of GridView.count()?
2. How does List.generate work in this Program?
3. What does crossAxisCount define?

Practical No- 16

Aim: To create a navigation drawer in a Flutter application.

Requirements:
• Flutter SDK
• Android Studio or VS Code
• Emulator or physical device
• Laptop or desktop system

Theory:

A navigation drawer is a panel that slides from the left side of the screen to provide access to different sections of the app.
It is commonly used in mobile apps for menus, settings, or user profile access.
Flutter provides the Drawer widget, which is added to the Scaffold's drawer property.
The Drawer usually contains a ListView with menu items, including a DrawerHeader for app title or user info.
ListTile widgets inside the drawer act as tappable menu items, which can later be linked to navigate to other screens.
The drawer can be opened by tapping the menu icon in the AppBar or swiping from the left edge.
Using a navigation drawer improves user accessibility and organizes the app navigation efficiently.

Algorithm:

1. Create a new Flutter project.
2. Create a StatelessWidget called DrawerApp.
3. Add a Scaffold with an AppBar.
4. Add a Drawer to the drawer property of the Scaffold.
5. Inside the Drawer, add a ListView with a DrawerHeader and ListTile items.
6. Add main content in the body property of Scaffold.
7. Run the app and open the drawer by tapping the icon or swiping.

Program:

Output:
Conclusion:
Question:

1. What is the purpose of the Drawer widget in Flutter?
2. How do you open a drawer in the app?
3. What is the role of ListTile inside a Drawer?